

De herramienta interna a producto

Qué falta para dejar de correr en un VPS con un túnel improvisado y poder cobrarle a una empresa externa por esto — infraestructura, equipo, soporte, legal y precio, en el orden en que hay que hacerlo.

19 de agosto de 2026 · un solo desarrollador · 84.477 líneas · 71 migraciones

01 Lo que ya tienes construido

Antes de hablar de plan, la foto real del código, porque define todo lo que sigue. Esto no es un prototipo: es un sistema grande, funcionando, con automatización propia.

CÓDIGO 84.477 líneas TypeScript/React en 311 archivos	MÓDULOS DE DOMINIO 110 librerías en <code>app/lib</code> (viabilidad, anexos, OCR, costeo, auditor)
ESQUEMA DE DATOS 71 migraciones SQL acumuladas	COBERTURA DE PRUEBAS 24 archivos de test + golden set de regresión

Lo que hace el producto, en lenguaje de venta

Cinco capacidades que sí son vendibles por separado y que hoy conviven en la misma app:

1. **Radar.** Busca en Mercado Público, filtra por rubro y palabras negativas, y avisa lo que le sirve a cada perfil.
2. **Análisis de viabilidad con IA.** Lee las bases, detecta modalidad de adjudicación, requisitos y riesgos, con citas a la página del PDF.
3. **Anexos.** Separa, clasifica y rellena los formularios del oferente — el módulo más diferenciador y el más difícil de copiar.
4. **Costeo.** Genera la planilla económica y busca precios de mercado por ítem.
5. **Auditor técnico y gestión.** Compara la oferta contra las bases línea por línea, más el pipeline de postuladas, adjudicadas y órdenes de compra.

La barrera de entrada que ya ganaste

Mercado Público bloquea por WAF toda descarga que no venga de una IP de ISP chileno. Vercel, AWS y los proxies comerciales no pasan. Tú resolviste eso con servidor propio en Chile. Esa es tu ventaja competitiva real y, al mismo tiempo, tu principal restricción de infraestructura: el producto no puede vivir en un cloud gringo.

02 Las cinco brechas que bloquean vender a un externo

Todo lo que sigue en este plan cuelga de aquí. Hoy el sistema está construido para una organización — la tuya. Estas son las cosas que hoy, tal cual, impiden entregarle acceso a una empresa que no seas tú.

BRECHA	ESTADO HOY	POR QUÉ BLOQUEA	ESFUERZO
Aislamiento por cliente	NO EXISTE	Los roles son admin / usuario / externo y la tabla empresas modela al cliente del negocio, no al inquilino del sistema. Dos empresas en la misma base se verían las licitaciones, los costos y los precios entre sí.	Alto
Base de datos productiva	HOSTING COMPARTIDO	MySQL en Bluehost: sin respaldo verificado, sin réplica, sin control de max_connections , latencia desde Chile a EE.UU. en cada consulta. Ya te costó ~18 s de radar por un choque de collation.	Medio
Dirección pública estable	TÚNEL TEMPORAL	Cloudflare Quick Tunnel entrega una URL aleatoria que cambia en cada reinicio. No se le puede dar a un cliente ni a un cron externo.	Bajo
Cobro y facturación	NO EXISTE	Cero rastro de pasarela de pago, planes o suscripciones en el código. Sin esto no hay negocio recurrente, solo proyectos a pedido.	Medio
Observabilidad y respaldo	NADA	Sin Sentry, sin /api/health , sin alertas, sin respaldos automatizados. Si el VPS cae un sábado, te enteras cuando el cliente te escribe el lunes.	Bajo

LA DECISIÓN QUE ORDENA TODO

El aislamiento por cliente define si esto es un SaaS o un software instalado. Y esa decisión cambia el VPS, el equipo, el soporte y el precio. Va primero, antes de escribir una línea.

03 Tres modelos de entrega. Elige uno.

Siendo un solo desarrollador, la diferencia entre estos tres caminos no es el ingreso: es cuánta carga operativa te echas encima. Ordenados de menor a mayor riesgo personal.

MODELO	QUÉ VENDES	TRABAJO DE INGENIERÍA	CARGA OPERATIVA	TECHO COMERCIAL
A · Instancia dedicada Single-tenant	Un contenedor y una base por cliente, en tu VPS. Cobras arriendo mensual + implementación.	Bajo. No tocas el modelo de datos. Automatizas el despliegue y listo.	Crece lineal: cada cliente es un despliegue que actualizar.	Realista hasta 8–12 clientes.
B · SaaS multi-cliente Multi-tenant	Una sola app, cuentas por empresa, planes y cobro automático.	Alto. <code>tenant_id</code> en cada tabla, en cada consulta y en cada job. Es la refactorización más cara del plan.	Plana. Un despliegue sirve a todos.	Sin techo técnico. Es el negocio grande.
C · Servicio con software Managed	Tú operas el sistema y entregas el resultado: informes, anexos llenos, costeo. El cliente no entra a la app.	Ninguno. Vendible <i>esta semana</i> .	Alta y sobre ti: tu tiempo es el producto.	Limitado por tus horas.

RECOMENDACIÓN

C para financiarte ahora, A como primer producto, B solo cuando A tenga 5 clientes pagando. Construir multi-tenant antes de tener clientes es el error clásico: seis meses de refactorización sin validar que alguien paga. El modelo A te deja cobrar en 6–8 semanas y reutiliza todo el `docker-compose` que ya tienes.

04 Infraestructura: qué VPS y cuándo cambiarlo

Tu stack es inusualmente pesado en memoria porque hace trabajo real en el servidor: Chromium con Puppeteer para el descargador de MP, LibreOffice headless para convertir `.doc`, Tesseract para OCR, y Next.js con el `build` completo. No es una app CRUD.

Restricción que no se negocia

El VPS **tiene que estar en Chile, con IP de ISP chileno**. Hetzner, DigitalOcean, AWS y Vercel quedan descartados para todo lo que toque el portal de MP. Proveedores válidos: V2Networks (el que ya usas), Netxcorp, HostingPlus, ZGlobalHost.

ETAPA	CLIENTES	SERVIDOR DE APLICACIÓN	BASE DE DATOS	COSTO MENSUAL ESTIMADO
Hoy	1 (interno)	4 vCPU · 8 GB RAM · 100 GB NVMe	Bluehost compartido MIGRAR	USD 25–45
Primer externo	1–5	8 vCPU · 16 GB RAM · 200 GB NVMe	MySQL gestionado o VPS propio: 2 vCPU · 4 GB	USD 90–150
Producto	5–20	App 8 vCPU/16 GB + worker aparte 8 vCPU/16 GB (Chromium, OCR y LibreOffice fuera de la app)	4 vCPU · 8 GB + réplica de lectura	USD 250–400
Escala	20+	2 nodos de app tras balanceador + pool de 2–3 workers	Gestionado con failover	USD 600–1.000

Cambios de infraestructura, en orden

1. **Dominio propio + túnel con nombre.** Compra el dominio, agrégalo a Cloudflare, cambia el Quick Tunnel por túnel con token. Una tarde de trabajo y elimina la URL que se mueve sola. Alternativa: Caddy con TLS automático apuntando a la IP del VPS.
2. **Sacar MySQL de Bluehost.** A un VPS chileno propio con `mysqldump` diario a Cloudflare R2 (que ya usas), o a un MySQL gestionado. Verifica la restauración una vez al mes — un respaldo que nunca se restauró no es un respaldo.
3. **Separar el worker.** Hoy el mismo contenedor sirve páginas y corre Chromium. Un OCR pesado hace lento el front. Mueve el trabajo largo a su propio servicio de compose con la cola que ya modelaste en `viabilidad-jobs` y `auditor-tecnico-jobs`.
4. **Gestor de secretos.** Hoy todo vive en un `.env` plano en el VPS: claves de Z.AI, ticket de MP, JWT, credenciales de base. Con externos eso ya no alcanza — mínimo Doppler o Infisical, o variables cifradas en el pipeline de despliegue.
5. **Ambiente de pruebas.** Hoy no existe: pruebas en producción. Un segundo compose en el mismo VPS con su propia base es suficiente y cuesta casi nada.
6. **Despliegue automático.** `git push` → GitHub Actions → `docker compose up -d --build` en el VPS. Reemplaza al `actualizar-servidor.bat` manual.

RIESGO DE PROVEEDOR

Cuatro dependencias externas pueden botar el producto y ninguna es tuya: el ticket de la API de Mercado Público, el WAF de MP (si cambia la protección, el descargador muere), Z.AI/GLM como motor de IA, y el propio VPS chileno. Para las dos de IA ya tienes respaldo (DeepSeek, Tesseract). Para las dos de MP, no hay. Eso hay que decírselo al cliente en el contrato, no descubrirlo juntos.

05 Equipo: a quién contratar, en qué orden

Eres el único desarrollador de 84.477 líneas. Comercialmente eso tiene un nombre: **factor bus igual a uno**. Cualquier cliente serio, o cualquier inversionista, lo va a preguntar. No es un detalle técnico, es una objeción de venta.

La primera contratación no es un desarrollador

Es la respuesta contraintuitiva pero es la correcta. Un segundo desarrollador tarda 3–4 meses en ser productivo en un sistema de este tamaño, y mientras tanto *te resta* tiempo. Lo que sí te devuelve horas de inmediato es alguien que absorba el soporte y la puesta en marcha de cada cliente.

ORDEN	ROL	CUÁNDO	DEDICACIÓN	COSTO MENSUAL BRUTO	QUÉ TE LIBERA
1º	Soporte y puesta en marcha	Con el 1er cliente externo	Media jornada	\$600k – \$900k	Capacitación, mesa de ayuda de primer nivel, carga de datos del cliente. Puede ser el mismo perfil comercial.
2º	Dev full-stack semi-senior	Con 3 clientes o al empezar multi-tenant	Jornada completa	\$1,8M – \$2,6M	Mantenición, front, correcciones. Tú te quedas con el motor de IA y anexos, que es lo que nadie más puede tomar rápido.
3º	DevOps / infraestructura	Al separar workers	Por horas o proveedor	\$500k – \$1,2M	Despliegues, respaldos, monitoreo, seguridad. Tercerizable — no necesitas a alguien en planilla.
4º	Comercial	Al validar el precio	Base + comisión	\$700k + var.	Prospección y cierre. Antes de tener precio validado, vender es trabajo tuyo.

Rangos referenciales de mercado chileno a agosto de 2026, brutos y sin costos patronales. Validar al momento de contratar.

Reducir el factor bus sin contratar

- **Documento de arquitectura de una página.** Los 11 `.md` en `docs/` son bitácoras de cambios, no un mapa. Falta el "cómo está armado esto" que un dev nuevo lee el día uno.
- **Migraciones ordenadas.** 71 archivos `aplicar-migration-NN.mjs` corridos a mano. Un solo comando `migrate` que sepa en qué versión está la base.

- **Ampliar las pruebas.** 24 archivos de test para 110 módulos. Prioriza lo que un tercero rompería sin darse cuenta: anexos, viabilidad, cálculo de costeo.
- **Depósito de código en garantía.** Para clientes grandes, un *escrow* — si desapareces, ellos acceden al código. Elimina la objeción de raíz y cuesta poco.

06 Soporte: qué prometes por escrito

Sin un acuerdo de nivel de servicio explícito, el cliente asume que estás disponible siempre. Con uno, tú defines las reglas. Escríbelo *antes* del primer contrato, no después del primer reclamo.

NIVEL	QUÉ ES	RESPUESTA	SOLUCIÓN	HORARIO
Crítico	Sistema caído, o cierre de licitación en riesgo	2 h hábiles	Mismo día	L-V 9-19
Alto	Un módulo no funciona, hay forma de rodearlo	1 día hábil	3 días hábiles	L-V 9-19
Normal	Error menor, consulta de uso	2 días hábiles	Próxima versión	L-V 9-19
Mejora	Función nueva	5 días hábiles	Se prioriza y cotiza	—

Lo mínimo para poder cumplirlo

- **Canal único.** Correo con mesa de ayuda simple, no WhatsApp. El WhatsApp del desarrollador no escala y no deja registro.
- **Monitoreo con aviso.** `/api/health` que verifique base, R2 y proveedor de IA, más un *uptime check* externo que te escriba a ti antes que el cliente.
- **Registro de errores.** Sentry o similar. Hoy los errores viven en el log del contenedor y nadie los mira. Ya identificaste el patrón: los `catch` mudos ocultan fallas reales.
- **Tablero de estado.** Una página pública de estado del servicio evita la mitad de los correos de "¿está caído?".
- **Compromisos honestos.** No prometas 99,9% con un VPS único: eso son 8 horas de caída al año y una actualización mal hecha te las gasta. Comprometé 99% y cúmplelo.

07 Legal, datos y contratos

Esta sección es la que más se salta y la que más caro sale. Son documentos, no código, y se hacen una vez.

- **Sociedad y facturación.** Si aún facturas como persona natural, una SpA separa tu patrimonio del riesgo del negocio y es lo que una empresa mediana espera ver en la orden de compra.
- **Datos personales.** Estás procesando RUT, representantes legales y documentos tributarios de terceros. Aplica la ley 19.628 y su reforma; necesitas un anexo de tratamiento de datos donde quede claro que el cliente es el responsable y tú el encargado.

- **Contrato de licencia y servicio.** Alcance, nivel de servicio, propiedad intelectual (**el código es tuyo**, el cliente recibe licencia de uso), datos del cliente son del cliente, límite de responsabilidad, y salida con exportación de datos.
- **Cláusula de IA.** El sistema entrega análisis generados por modelos de lenguaje. Deja escrito que son apoyo a la decisión y que la responsabilidad de la oferta presentada es del cliente. Es tu protección más importante y cuesta un párrafo.
- **Términos de Mercado Público.** Revisa las condiciones de uso de la API y del portal antes de vender acceso automatizado a terceros. Es el punto legal con más incertidumbre y conviene resolverlo con un abogado, no por analogía.
- **Marca.** Registra LICITANK en INAPI, clase 42 (servicios de software) y clase 35 si asesoras comercialmente.

08 Precio: primero mide, después cobra

No inventes un precio. Tienes algo mejor: **el sistema ya mide su propio costo**. La migración 71 guarda llamadas, tokens de entrada y salida y costo en dólares por corrida del auditor, y el mismo acumulador existe para viabilidad. Úsalo.

Lo que hay que medir antes de cotizar

1. Costo de IA por licitación analizada completa: viabilidad + anexos + costeo + auditor.
2. Cuántas licitaciones al mes procesa un cliente típico.
3. Costo de infraestructura repartido entre clientes.
4. Horas tuyas de puesta en marcha por cliente nuevo.

Con esos cuatro números tienes el piso. Regla práctica: el precio de lista no baja de **4 a 5 veces el costo variable**, porque tiene que pagar soporte, desarrollo y los clientes que no cierran.

Estructura sugerida

- **Implementación, pago único.** Cubre carga de datos de la empresa, sus anexos, capacitación y ajuste de perfiles de búsqueda. Esto es trabajo real y no puede ir gratis.
- **Suscripción mensual por empresa**, con un tope de licitaciones analizadas incluidas y precio por análisis adicional. El tope es lo que te protege del costo de IA.
- **Éxito, opcional.** Un porcentaje sobre licitaciones adjudicadas. Alinea intereses y es muy vendible, pero exige que puedas comprobar la adjudicación — y ya la lees desde la API de MP.

EL ARGUMENTO DE VENTA

No vendas "software de licitaciones". Vende dos cosas medibles: las horas que un analista deja de gastar rellenando anexos a mano, y las licitaciones que se dejaron de perder por un requisito no visto. **Una sola licitación ganada paga años de suscripción** — y tus propios datos ya lo demuestran: 16 ganadas por \$409M con 57% de éxito.

09 El plan, fase por fase

Estimaciones para ti trabajando solo. Las fases son secuenciales porque cada una depende de la anterior; lo único paralelizable es lo legal, que no requiere código.

FASE 0

2–3 semanas

Ponerlo firme

Sin clientes todavía. El objetivo es que el sistema deje de depender de que tú lo estés mirando.

- Dominio propio y túnel con nombre — se acaba la URL que cambia sola.
- MySQL fuera de Bluehost, con respaldo diario a R2 y una restauración probada.
- `/api/health`, Sentry y un *uptime check* externo.
- Despliegue por GitHub Actions y ambiente de pruebas separado.
- Secretos en un gestor, fuera del `.env` plano.

Hito: puedes irte una semana sin que nada se caiga en silencio.

FASE 1

3–4 semanas

Hacerlo entregable

Modelo A: una instancia por cliente. Aquí aparece el primer peso.

- Instalación por script: un comando levanta cliente nuevo con su base y su compose.
- Personalización mínima por cliente: logo, datos de empresa, perfiles de búsqueda.
- Manual de usuario y capacitación grabada — lo que hoy explicas en vivo, una sola vez.
- Contrato, acuerdo de nivel de servicio y anexo de datos personales, revisados por abogado.
- Medición del costo real de IA por licitación, con el instrumental que ya existe.

Hito: primer cliente externo pagando implementación + mensualidad.

FASE 2

2-3 meses

Que soporte a varios

Entre 2 y 5 clientes. Aquí duele la operación, no el código.

- Contratar soporte de media jornada.
- Separar el worker: Chromium, OCR y LibreOffice fuera del contenedor de la app.
- Mesa de ayuda con registro de tickets.
- Panel interno: qué cliente consume cuánto, cuánto cuesta cada uno.
- Ampliar pruebas en anexos, viabilidad y costeo.

Hito: ingreso recurrente que cubre infraestructura + soporte.

FASE 3

4-6 meses

Convertirlo en producto

Solo si la Fase 2 llegó a 5 clientes pagando. Si no llegó, el problema es comercial y multi-tenant no lo arregla.

- Multi-tenant real: `tenant_id` en el esquema, en cada consulta y en cada job en segundo plano.
- Registro autoatendido, planes y pasarela de pago.
- Contratar el segundo desarrollador.
- Escalar infraestructura: réplica de lectura, pool de workers, balanceador.

Hito: un cliente puede registrarse, pagar y usar el sistema sin que tú intervengas.

10 Qué hacer esta semana

Cinco cosas concretas, en orden, ninguna toma más de un día.

1. **Decide el modelo.** A, B o C. Todo lo demás depende de esta respuesta y es tuya, no técnica.
2. **Compra el dominio y monta el túnel con nombre.** Es la mejora con mejor relación esfuerzo/resultado de todo el plan.
3. **Saca la base de Bluehost** y verifica que un respaldo se puede restaurar. Este es el único punto donde una falla es irreversible.
4. **Corre el auditor sobre 10 licitaciones y anota el costo en dólares.** El instrumental ya está; sin ese número no puedes cotizar.
5. **Anda a hablar con tres empresas del rubro.** No para vender: para preguntar cuánto tiempo pierden hoy y cuánto pagarían. Eso vale más que dos meses de código.

EL RIESGO MÁS GRANDE NO ES TÉCNICO

El sistema está mucho más maduro de lo que suele estar un producto en esta etapa. El riesgo es gastar los próximos seis meses construyendo multi-tenant, pagos y paneles para un mercado que todavía no confirmaste que paga. Valida el precio con clientes reales antes de la Fase 3.

Basado en el estado del repositorio al 19 de agosto de 2026. Los rangos de costo de infraestructura y remuneraciones son referenciales y deben validarse con cotizaciones al momento de contratar.